Parallel Locally Optimal Iterative Methods

SIAM Conference on
Parallel Processing for Scientific Computing
Zuse Institute & Free University of Berlin

Nicolas Venkovic, Amir Bouslama,
Aryaman Jeendgar and Hartwig Anzt

Chair of Computational Mathematics
School of Computation, Information and Technology
Technical University of Munich

March 5, 2026

Outline I

1. Introduction 1
2. Locally optimal approximation of eigenpairs 2
3. Locally optimal approximation of matrix inverses 9
4. Locally optimal low-rank matrix approximation 19
5. Closing remarks 22

Introduction

The "What?", the "Why?" and the "How?"

- Numerical linear algebraic problems of interest include:
  - Finding eigenpairs of matrices $A, B \in \mathbb{R}^{n\times n}$:

$$\boxed{(\lambda, x) \in \mathbb{C} \times \mathbb{C}^n \setminus \{0\} \text{ s.t. } Ax = \lambda Bx}$$

  - Finding the inverse of a matrix $A \in \mathbb{R}^{n\times n}$:

$$\boxed{M \in \mathbb{R}^{n\times n} \text{ s.t. } AM = I_n}$$

  - Finding low-rank factorizations of a matrix $X \in \mathbb{R}^{m\times n}$:

$$\boxed{(U, V) \in \mathbb{R}^{m\times r} \times \mathbb{R}^{n\times r} \text{ s.t. } UV^T = X \text{ for some } r \leq \min(m, n)}.$$

- Iterative methods are often the preferred approach for very large problems, i.e., for large values of m and/or n.
- Iterative methods rely on recurrence relations which can be short, or longer.
- **Short-recurrence iterative methods** can exhibit **slow convergence**.
- Short-recurrence iterative methods can be made "**locally optimal**" by expanding the recurrence length (i.e., enriching the search space).

Locally optimal approximation of eigenpairs

First-order approximation of eigenpairs

- For all symmetric matrix pencils (A, B) with symmetric positive definite (SPD) matrix B, every eigenpair $(\rho(x), x)$ such that $Ax = \rho(x)Bx$ is a stationary point of the Rayleigh quotient

$$\rho(x) := \frac{x^T A x}{x^T B x} \text{ with } \nabla\rho(x) \propto r := Ax - \rho(x)Bx$$

where r denotes the eigen-residual.

- Given an initial iterate x_0, a dominant (min or max) eigenvector may be solved by steepest descent (SD) iteration:

$$x_{i+1} := \arg\left(\min \text{ or } \max\right)_{x \in x_i + \text{span}\{r_i\}} \rho(x) \text{ for } i = 0, 1, \ldots$$

which is obtained by the orthogonal projection:

$$x_{i+1} \in x_i + \text{span}\{r_i\} \text{ s.t. } r_{i+1} := Ax_{i+1} - \rho(x_{i+1})Bx_{i+1} \perp \text{span}\{r_i\}$$

for $i = 0, 1, \ldots$

First-order approximation of eigenpairs, cont'd

- SD iterations often converge slowly.
  For faster convergence, alternative short-recurrences include conjugate gradient iterations:

$$x_{i+1} \in x_i + \text{span}\{p_i\} \;\; \text{where} \;\; p_i \in r_i + \text{span}\{p_{i-1}\}$$

  denote conjugate search directions, that is, we have

$$p_{i+1} \perp Ap_i \;\; \text{for} \;\; i = 0, 1, \ldots \;\; \text{with} \;\; p_0 := r_0.$$

  Since the Rayleigh quotient is generally not quadratic, different update formulas exist that fit this configuration, see Feng and Owen (1996).
- In practice, more than one left or right-most eigenpairs are computed simultaneously.
  That is, we work with an iterate $X \in \mathbb{R}^{n \times k}$ of k approximate eigenvectors to minimize the Rayleigh quotient

$$\rho(X) := \frac{\text{tr}(X^T A X)}{\text{tr}(X^T B X)}.$$

Feng, Y. T., & Owen, D. R. J. (1996). Conjugate gradient methods for solving the smallest eigenpair of large symmetric eigenvalue problems. International Journal for Numerical Methods in Engineering, 39(13), 2209-2229.

Approximation of eigenpairs in Krylov subspaces

- Eigenpairs are commonly approximated in Krylov subspaces of the form

$$\mathcal{K}_i(A, x_0) := \text{span}\{x_0, Ax_0, \ldots, A^{i-1}x_0\}.$$

  For a general matrix A, the construction of an orthogonal basis for the space $\mathcal{K}_i(A, x_0)$ relies on long recurrence relations.

  When A is symmetric, in exact arithmetic, those relations simplify to three-terms recurrences, thus reducing memory requirements, traffic and computation. This is the Lanczos procedure.

  However:

  - In finite precision, the basis vectors lose their orthogonality, sometimes requiring fixing through longer recurrences.
  - For generalized eigenvalue problems ($B \neq I_n$), one either needs to (i) apply a factorization of B, or (ii) solve a linear system with B.
- Alternatively, first-order short recurrences are made "locally optimal" to
  - Improve convergence,
  - Bypass the need for significantly longer recurrences and solves with B.

Locally optimal block conjugate gradient (LOBPCG)

- The term "locally optimal" was coined by Knyazev (1991, 2001) to refer to the enrichment of the search space used for eigenpair approximation by conjugate gradient (CG).

  To form an iterate x_{i+1}, Knyazev uses the search space

  $$\text{span}\{x_i, r_i, p_{i-1}\} \text{ instead of } \text{span}\{x_i, r_i + \alpha p_{i-1}\}$$

  where the latter is the search space of CG iterations for some step size α.

  Optimizing the Rayleigh quotient $\rho(x_{i+1})$ over a richer search space can significantly enhance convergence.

  Knyazev forms eigenpair approximations by Rayleigh-Ritz projection. That is, the iterate $x_{i+1} \in \text{span}\{x_i, r_i, p_{i-1}\}$ is set such that

  $$r_{i+1} := Ax_{i+1} - \rho(x_{i+1})Bx_{i+1} \perp \text{span}\{x_i, r_i, p_{i-1}\}.$$

Knyazev, A. V. (1991). A preconditioned conjugate gradient method for eigenvalue problems and its implementation in a subspace. In Numerical Treatment of Eigenvalue Problems Vol. 5/Numerische Behandlung von Eigenwertaufgaben Band 5: Workshop in Oberwolfach, February 25–March 3, 1990/Tagung in Oberwolfach, 25. Februar–3. März 1990 (pp. 143-154). Birkhäuser Basel.

Knyazev, A. V. (2001). Toward the optimal preconditioned eigensolver: Locally optimal block preconditioned conjugate gradient method. SIAM journal on scientific computing, 23(2), 517-541

Locally optimal block conjugate gradient (LOBPCG), cont'd

- LOBPCG refers to the case where
  - a block $X_i \in \mathbb{R}^{n\times k}$ of k approximate eigenvectors are simultaneously iterated, and
  - the residual block $R_i := AX_i - BX_i\Lambda_i$ is preconditioned.

  That is, $X_{i+1} := [x_1, \ldots, x_k]$ is formed by Rayleigh-Ritz projection:

  $$x_\ell \in \text{range}(V) \;\; \text{s.t. } r_\ell := Ax_\ell - \rho(x_\ell)Bx_\ell \perp \text{range}(V) \;\; \text{for} \;\; \ell = 1, \ldots, k$$

  with $V := [X_i, Z_i, P_{i-1}]$ where $Z_i := TR_i$ is the block of preconditioned residuals.

  In the Rayleigh-Ritz projection, the factorization of V^TBV may fail. Remedies include:
  - Full B-orthonormalization of V (Hetmaniuk and Lehoucq, 2006),
  - B-orthonormalization of colum blocks of V (Knyazev et al., 2007),
  - Full B-orthonormalization of V when needed (Duersch et al., 2018).

Hetmaniuk, U., & Lehoucq, R. (2006). Basis selection in LOBPCG. Journal of Computational Physics, 218(1), 324-332.

Knyazev, A. V., Argentati, M. E., Lashuk, I., & Ovtchinnikov, E. E. (2007). Block locally optimal preconditioned eigenvalue Xolvers (BLOPEX) in Hypre and PETSc. SIAM Journal on Scientific Computing, 29(5), 2224-2239.

Duersch, J. A., Shao, M., Yang, C., & Gu, M. (2018). A robust and efficient implementation of LOBPCG. SIAM Journal on Scientific Computing, 40(5), C655-C676.

Experimental results

- Effect of orthogonalization (and lack thereof) on convergence: Approximation of 10 out of 200 leftmost eigenpairs.
  - Julia implementations

Matrix	Basic	BLOPEX	IterativeSolvers.jl	Ortho	Skip_ortho
bcsstk15	✗	✗	✗	✓	✓
bcsstk21	✗	✗	✗	✓	✓
bodyy6	✗	✗	✗	✓	✓
tmt_sym	✗	✓	✘	✓	✓

✗: fails in the Rayleigh-Ritz procedure when trying to factorize the cross-product matrix $V^T V$.

✘: does not converge with wanted backward error (10^{-5}).

✓: converges with wanted backward error (10^{-5}).

  - Python implementations

Matrix	Basic	BLOPEX	SciPy	Ortho	Skip_ortho
bcsstk15	✗	✗	✘	✓	✓
bcsstk21	✗	✗	✓	✓	✓
bodyy6	✗	✗	✘	✓	✓
tmt_sym	✗	✓	✘	✓	✓

The matrices can be found in the SuiteSparse collection.

Experimental results, cont'd

- Multi-threaded runtimes:

  Approximation of 10 out of 200 leftmost generalized eigenpairs.

  Backward error of 10^{-5}, `num_threads` $= 6$.

Matrix pencil	`Basic`	`BLOPEX`	`IterativeSolvers.jl`	`Ortho`	`Skip_ortho`
`Poisson8`†	16 s 27 it	16 s 27 it	26 s 28 it	29 s 27 it	23 s 27 it
`Poisson16`†	✗	36 s 42 it	57 s 42 it	73 s 41 it	48 s 41 it
`Poisson32`†	✗	82 s 55 it	132 s 55 it	188 s 54 it	108 s 54 it
`Poisson64`†	✗	249 s 89 it	423 s 89 it	640 s 88 it	378 s 88 it
`Poisson128`†	628 s 102 it	761 s 102 it	1253 s 103 it	2038 s 102 it	1199 s 102 it

Experiences done on an Intel CPU 12th Gen Core i7-1255U.

†: `PoissonX` refers to a matrix pencil from the unstructured FE discretization of a 2D Poisson PDE with a random variable coefficient s.t. $n(A) \approx \mathtt{X} \times 1000$.

✗: fails in the Rayleigh-Ritz procedure when trying to factorize the cross-product matrix $V^T V$.

The matrices can be found at `github.com/venkovic/matrix-market`.

Locally optimal approximation of matrix inverses

Approximation of matrix inverses

- For (right) approximate matrix inversion, we search for a matrix M s.t.

$$f(M) := \|I_n - AM\|_F^2 \text{ with } \nabla_M f(M) \propto A^T R$$

is minimized, where $R := I_n - AM$ denotes the residual.

- Steepest descent (SD) iterations are defined by the recurrence formula

$$M_{i+1} := \arg \min_{M \in M_i + \mathsf{span}\{A^T R_i\}} f(M)$$

which is equivalently defined by the following projection:

$$M_{i+1} \in M_i + \mathsf{span}\{A^T R_i\} \text{ s.t. } R_{i+1} := I_n - AM_{i+1} \perp A\,\mathsf{span}\{A^T R_i\}.$$

- Faster convergence is achieved by minimal residual (MR) iteration:

$$M_{i+1} := \arg \min_{M \in M_i + \mathsf{span}\{R_i\}} f(M)$$

which is achieved by the following projection:

$$M_{i+1} \in M_i + \mathsf{span}\{R_i\} \text{ s.t. } R_{i+1} := I_n - AM_{i+1} \perp A\,\mathsf{span}\{R_i\}.$$

Chow, E., & Saad, Y. (1998). Approximate inverse preconditioners via sparse-sparse iterations. SIAM Journal on Scientific Computing, 19(3), 995-1023.

Approximation of matrix inverses, cont'd

- The vectorization of the right matrix inverse equation ($AM = I_n$) leads to

$$(I_n \otimes A)\,\mathrm{vec}(M) = \mathrm{vec}(I_n)$$

which can be used as a starting point for the definition short-recurrence iterations over Krylov subspace of the form

$$\mathrm{vec}(M_i) \in \mathrm{vec}(M_0) + \mathcal{K}_i(I_n \otimes A, \mathrm{vec}(M_0))$$

or, alternatively,

$$M_i \in M_0 + \mathcal{K}_i(A, M_0)$$

using Frobenius inner products.

Then, for an SPD matrix A, every new CG iterate requires the evaluation of

$$\|R_i\|_F^2 \text{ and } (P_i, AP_i)_F.$$

For general matrices, BiCGStab iterates are considered.

Locally optimal minimal residual (LOMR)

- Analogously to the way LOBPCG is introduced, in LOMR (Venkovic and Anzt, 2025), a new iterate M_{i+1} using the search space

$$\text{span}\{M_i, R_i, P_{i-1}\} \;\text{ instead of }\; \text{span}\{M_i, R_i\}$$

where the latter is the search space of MR iterations for some step size α. Given $A \in \mathbb{R}^{n\times n}$ and $M_0 \in \mathbb{R}^{n\times n}$, a sequence of LOMR iterates is defined by:

$$M_{i+1} := \arg \min_{M \in M_i + \text{span}\{P_{i-1}, R_i\}} \|I_n - AM\|_F \;\text{ for }\; i = 0, 1, \ldots.$$

The main iterates are given by:

$$M_{i+1} := M_i + \underbrace{\delta_i R_i + \gamma_i P_{i-1}}_{P_i} \;\text{ for }\; i = 0, 1, \ldots$$

where the optimal step sizes δ_i and γ_i depend on:

$$(R_i, AR_i)_F, \|AR_i\|_F^2, \|AP_{i-1}\|_F^2, (AR_i, AP_i)_F, (R_i, AP_i)_F.$$

N. Venkovic and H. Anzt (2025) Global iterative methods for sparse approximate inverses of symmetric positive-definite matrices. arXiv preprint, arXiv:2511.09753.

Summary of operations per iteration

- Each method requires four types of operations per iterate:

Method	Operation count per iteration
MR	1 (Sp)GEMM† + 1 (Sp)FIP$^\circ$ + 1 (Sp)FN$^\bullet$ + 2 (Sp)GEAMs*
SD	2 (Sp)GEMMs + 1 (Sp)FIP + 1 (Sp)FN + 2 (Sp)GEAMs
CG	1 (Sp)GEMM + 1 (Sp)FIP + 1 (Sp)FN + 3 (Sp)GEAMs
LOMR	2 (Sp)GEMMs + 3 (Sp)FIPs + 2 (Sp)FNs + 5 (Sp)GEAMs

†: General product of (sparse) matrices $^\circ$: Frobenius inner product of (sparse) matrices

$^\bullet$: Frobenius norm of (sparse) matrix *: General addition of (sparse) matrices

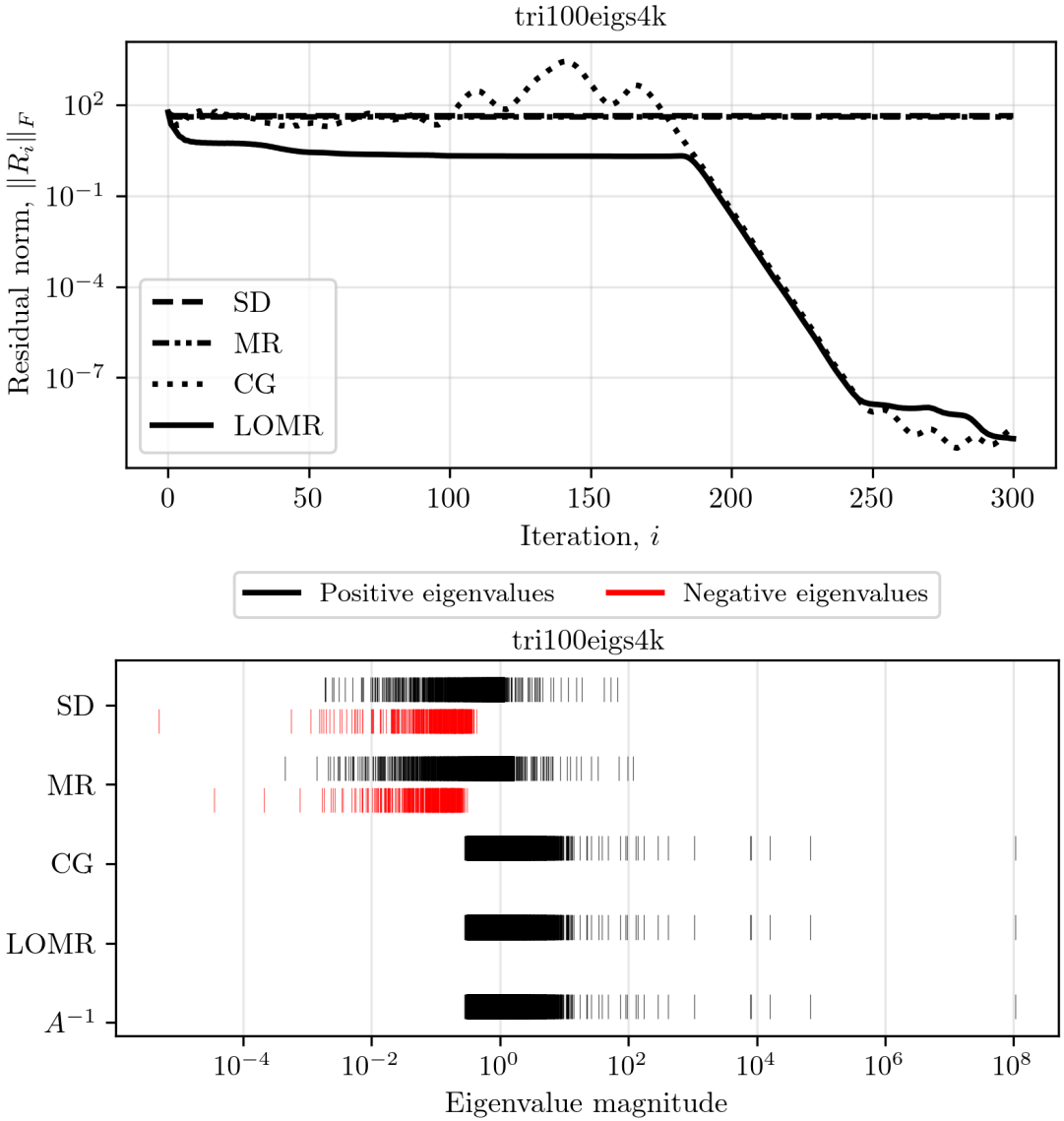
- Case of matrices with non-zero dropping:

Method	Operation count per iteration
MR	2 SpGEMM + 1 SpFIP + 1 SpFN + 2 SpGEAMs
SD	3 SpGEMMs + 1 SpFIP + 1 SpFN + 2 SpGEAMs
CG	2 SpGEMMs + 1 SpFIP + 1 SpFN + 3 SpGEAMs
LOMR	3 SpGEMMs + 3 SpFIPs + 2 SpFNs + 4 SpGEAMs

A substantial added cost is induced by dropping zeros and changing the data structures of the sparse iterates.

Comparison of methods

- Iterations with a Jacobi preconditioner:

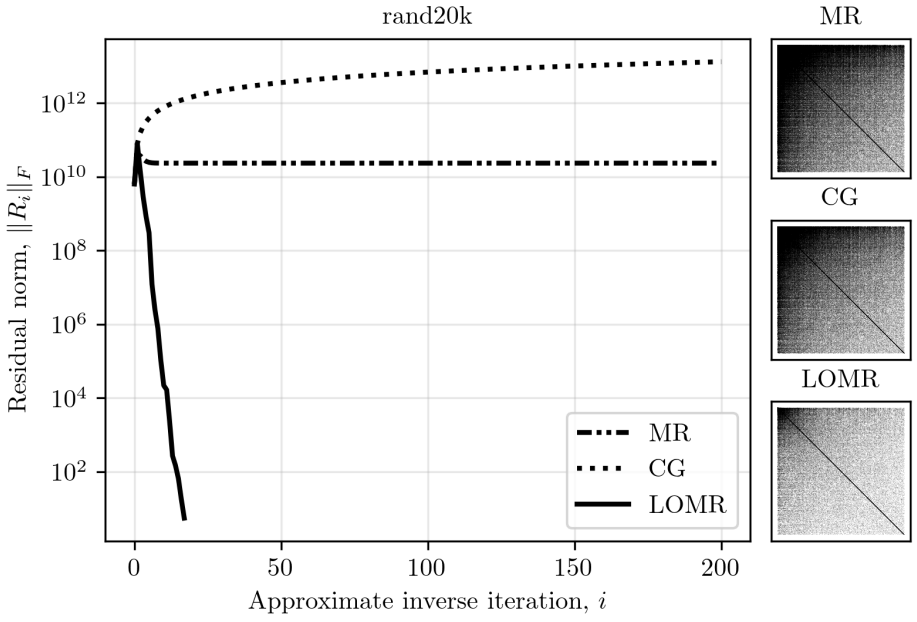


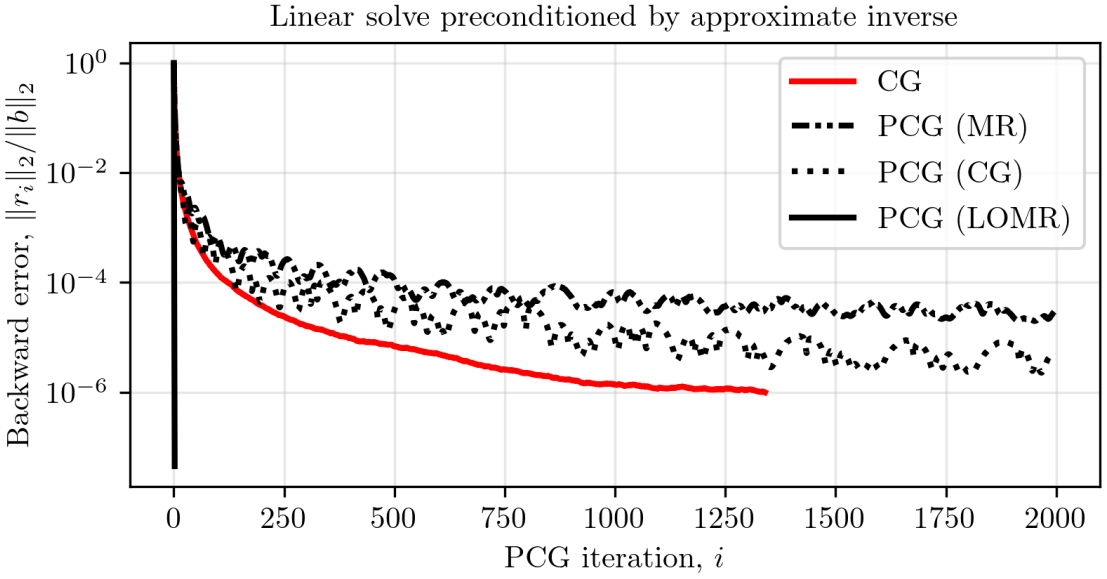
- SD and MR do not converge.
- CG and LOMR both converge.
- CG's convergence is not monotone.

  Strong dependence on spectrum.
- LOMR's convergence is monotone.

The matrix can be found at `github.com/venkovic/matrix-market`.

Comparison of methods, cont'd





The matrix can be found at `github.com/venkovic/matrix-market`.

Column-based variants for sparse approximate inverses

- Solving for the right approximate inverse M of A is embarrassingly parallelizable, i.e., we have

$$M := [m_1, \ldots, m_n] \text{ where } A\,m_j = e_j \text{ for } j = 1, \ldots, n.$$

Chow and Saad (1998) propose to self-precondition those solves:

Algorithm 1 Sequential self-preconditioned MR(A, M)

1: **for** $j = 1, \ldots, n$ **do**
2: Iteratively solve for m_j s.t. $A\,m_j \approx e_j$ using M as preconditioner
3: $M[:, j] := m_j$

Each column solve needs the solve of all previous columns to proceed.

For sparse approximation with non-zero dropping, sparse mode Krylov methods can be used in step 2, but MR have become the standard.

Chow, E., & Saad, Y. (1998). Approximate inverse preconditioners via sparse-sparse iterations. SIAM Journal on Scientific Computing, 19(3), 995-1023.

Benzi, M., & Tuma, M. (1999). A comparative study of sparse approximate inverse preconditioners. Applied Numerical Mathematics, 30(2-3), 305-340.

Column-based variants for sparse approximate inverses, cont'd$_1$

- To make the approach parallelizable, Chow and Saad (1998), introduce outer iterations with fixed preconditioning:

Algorithm 2 Parallel self-preconditioned MR(A, M_0)

1: **for** $i_o = 0, 1, 2, \ldots$ **do**
2: **for** $j = 1, \ldots, n$ **do**
3: Iteratively solve for m_j s.t. $A\,m_j \approx e_j$ using M_{i_o} as preconditioner
4: $M_{i_o+1}[:, j] := m_j$

where, once again, a sparse-mode Krylov solver can be used in place of MR in step 3.

Chow, E., & Saad, Y. (1998). Approximate inverse preconditioners via sparse-sparse iterations. SIAM Journal on Scientific Computing, 19(3), 995-1023.

Benzi, M., & Tuma, M. (1999). A comparative study of sparse approximate inverse preconditioners. Applied Numerical Mathematics, 30(2-3), 305-340.

Column-based variants for sparse approximate inverses, cont'd$_2$

- When using MR as an inner solver, the implementation is as follows:

Algorithm 3 Parallel self-preconditioned MR(A, M_0)

```
for i_o = 0, 1, 2, ... do
    for j = 1, ..., n do
        s := M_{i_o} e_j
        for i_i = 1, ..., n_i do
            r := e_j − As
            z := Mr
            q := Az
            α := (r, q)/(q, q)
            s := s + αz                    ▷ Non-zero droppings
    M_{i_o+1}[:, j] := s
```

Typically, only a few inner iterations (n_i) are used.

Chow, E., & Saad, Y. (1998). Approximate inverse preconditioners via sparse-sparse iterations. SIAM Journal on Scientific Computing, 19(3), 995-1023.

Column-based variants for sparse approximate inverses, cont'd$_3$

- Alternatively, we use LOMR as an inner solver:

Algorithm 4 Parallel self-preconditioned LOMR(A, M_0)

```
for i_o = 0, 1, 2, ... do
    for j = 1, ..., n do
        s := M_{i_o} e_j; r := e_j − As; p := 0
        for i_i = 1, ..., n_i do
            z := Mr; q := Az, w := Mp; v := Aw
            if i_i = 0 then
                δ := (r, q)/(q, q); γ := 1
            else
                c := (q, q) · (v, v) − (q, v)^2
                δ := [(v, v) · (r, q) − (q, v) · (r, v)] / c
                γ := [(q, q) · (r, v) − (q, v) · (r, q)] / c
            s := s + δz + γp; r := e_j − As; p := z + (γ/δ)p   ▷ Non-zero droppings
    M_{i_o+1}[:, j] := s
```

Chow, E., & Saad, Y. (1998). Approximate inverse preconditioners via sparse-sparse iterations. SIAM Journal on Scientific Computing, 19(3), 995-1023.

Locally optimal low-rank matrix approximation

Low-rank matrix approximation by alternating minimization

- For low-rank matrix approximation, we search for tall-and-skinny matrices U and V such that

$$g(UV^T) := \|X - UV^T\|_F^2 \quad \text{with} \quad \begin{cases} \nabla_U g(UV^T) \propto RV \\ \nabla_V g(UV^T) \propto R^T U \end{cases}$$

  is minimized, where $R := X - UV^T$ denotes the residual.
- Alternating minimization is commonly used to compute such approximations starting from an initial guess $U_0 V_0^T$:

$$U_{i+1} := \arg\min_{U \in \mathbb{R}^{n \times r}} g(UV_i^T) \text{ and } V_{i+1} := \arg\min_{V \in \mathbb{R}^{m \times r}} g(U_{i+1}V^T) \quad \text{for } i = 0, 1, \ldots$$

- Alternatively, block coordinate descent (BCD) iterations can be formed as follows from initial guesses U_0 and V_0:

$$\begin{cases} U_{i+1} := \arg\min_{U \in U_t + \text{span}\{R_i V_i\}} g(UV_i^T) \\ V_{i+1} := \arg\min_{V \in V_t + \text{span}\{\tilde{R}_i^T U_i\}} g(U_{i+1}V^T) \end{cases} \quad \text{for } i = 0, 1, \ldots$$

  where $\tilde{R}_i := X - U_{i+1}V_i^T$.

Locally optimal block coordinate descent (LOBCD)

- An alternative to BCD is introduced by considering richer search spaces. That is, in LOBCD, the low-rank factor iterates are such that

$$\begin{cases} U_{i+1} \in V_i + \text{span}\{P_{i_1}, R_i V_i\} \\ V_{i+1} \in V_i + \text{span}\{Q_{i-1}, R_i^T U_i\} \end{cases} \quad \text{instead of} \quad \begin{cases} U_{i+1} \in V_i + \text{span}\{R_i V_i\} \\ V_{i+1} \in V_i + \text{span}\{R_i^T U_i\} \end{cases}$$

where $P_i \in \mathbb{R}^{m \times r}$ and $Q_i \in \mathbb{R}^{n \times r}$ are search directions which arise in update formulas of the form

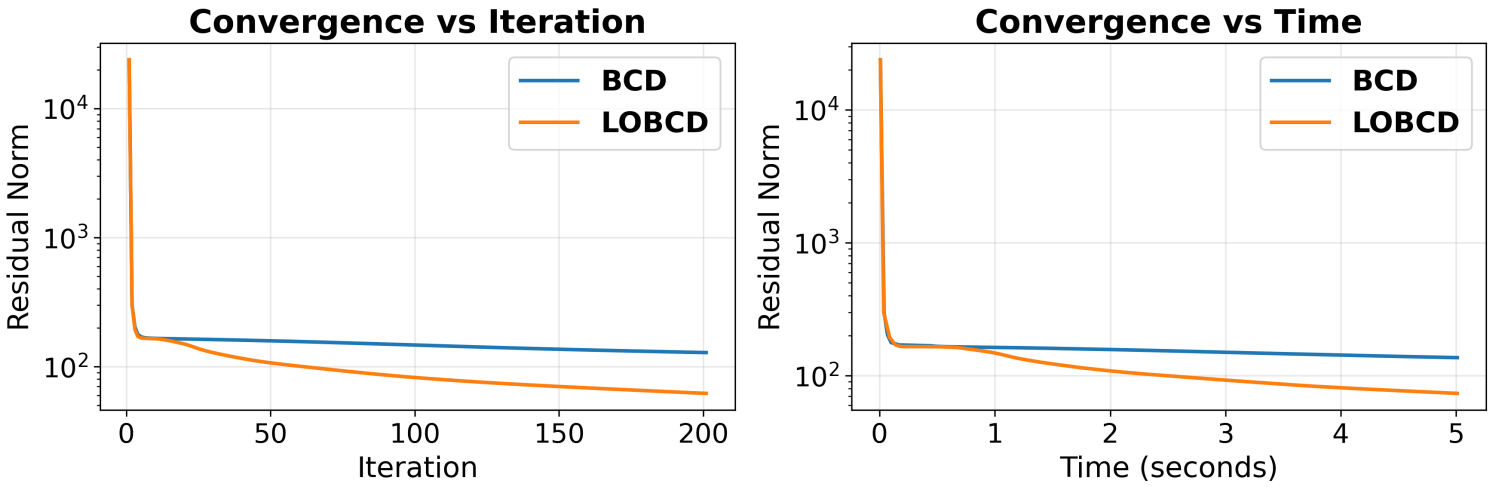
$$\begin{cases} U_{i+1} := U_i + \overbrace{\alpha_i R_i V_i + \beta_i P_{i-1}}^{P_i} \\ V_{i+1} := V_i + \underbrace{\gamma_i \tilde{R}_i^T U_{i+1} + \omega_i Q_{i-1}}_{Q_i} \end{cases}$$

where the computation of optimal step sizes $\alpha_i, \beta_i, \gamma_i$ and ω_i requires the evaluation of:

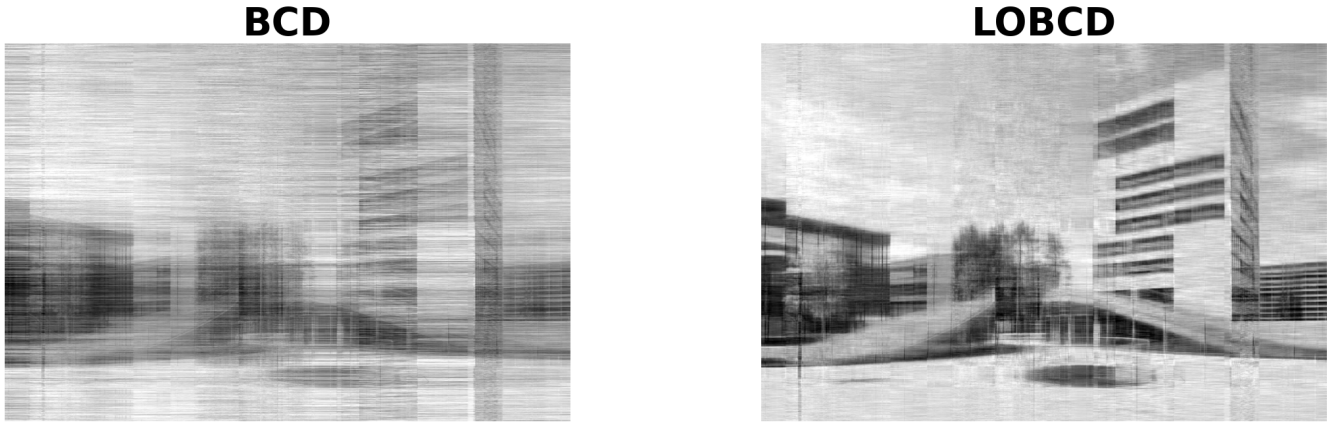
$$\|R_i V_i\|_F^2,\ (R_i V_i V_t^T V_i, R_i V_i)_F, (P_{i-1} V_i^T V_i, P_{i-1})_F,\ (R_i V_i V_i^T V_i, P_{i-1})_F,$$
$$(R_i V_i, P_{i-1})_F, \|\tilde{R}_i^T U_{i+1}\|_F^2,\ (\tilde{R}_i^T U_{i+1} U_{i+1}^T U_{i+1}, \tilde{R}_i^T U_{i+1})_F$$
$$(Q_{i-1} U_{i+1}^T U_{i+1}, Q_{i-1})_F,\ (\tilde{R}_i^T U_{i+1} U_{i+1}^T U_{i+1}, Q_{i-1})_F,\ (\tilde{R}_i^T U_{i+1}, Q_{i-1})_F.$$

Comparison of methods

- Rank-100 approximation of a 800-by-1,200 grayscale image:



- Low-rank approximations achieved in 5 seconds:



Closing remarks

Conclusion

- Take-away messages:
  - Locally optimal variants of short-recurrence iterative methods can significantly enhance convergence.
  - Column-oriented variants of locally optimal iterative methods, when possible, can further improve convergence.
- Dissemination:
  - Find this presentation at:

    venkovic.github.io/research

  - Code repositories:

    github.com/venkovic/ApproximateMatrixInverses.jl
    github.com/venkovic/StructuredLowRankMatrices.jl
    github.com/venkovic/julia-lobpcg
    github.com/venkovic/py-lobpcg

  - Preprint:

    N. Venkovic and H. Anzt (2025) Global iterative methods for sparse approximate inverses of symmetric positive-definite matrices. arXiv preprint, arXiv:2511.09753.

What's next ?

- Related ongoing work:
  - Parallelization of block coordinate descents for low-rank approximation.
  - Structured (sparse, non-negative) low-rank matrix approximation.
  - Application to structured matrix recovery.
  - Low-rank tensor approximation.
- Related future work:
  - Mixed precision variants.
  - Triangular factorization update.
- Next talk:
  - Petrov-Galerkin approximations in (multi)-linear algebra from the lens of optimization.

June 2 @ SIAM OP26, University of Edinburgh.